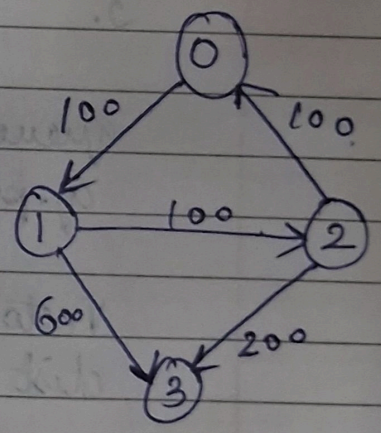
Experiment -7

There are n cities connected by some no. of flights. You are given an array flights where flights [i] = [from, to, price] indicates that there is a flight from city from to city to with cost price.

You are also given three integers src, dst and k return the cheapest price from src to dst with at most k stops. If there is no such route return -1.

I/p : n = 4, flights = [[0,1,100], [1,2,100], [2,0,100], [1,3,600], [2,3,200]], src = 0, dst = 3, k = 1

O/p : 700



⇒ Approach :

1) Build graph (adjacency list)

2) Use queue
Store (node, cost, stops)

3) Start
(src, 0, 0)

4) Traverse
For each node :
- if stops > k → skip
- Explore neighbours
- Update cost if cheaper.

5) Answer

- If destination reached → return cost.
- Else → return -1

Code:

```
# include <bits/stdc++.h>
using namespace std;

int findCheapestPrice (int n, vector<vector<int>>&
flights, int src, int dst, int k) {

    vector <vector <pair <int, int>>> adj (n);
    for (auto f : flights) {
        adj [f[0]]. push_back ({f[1], f[2]});
    }.

    queue <tuple < int, int, int >> q;
    q.push ({src, 0, 0});

    Vector <int > dist (n, INT_MAX);
    dist[src] = 0;

    while (!q. empty ()) {
        auto [node, cost, stops ] = q.front();
        q.pop();

        if (stops >k) continue;

        for (auto [next, price] : adj[node]) {
            if (cost + price < dist [next]) {
                dist[next] = cost + price;
```

```
            q.push({next, dis[next], stops+1});
        }
    }
}

    return dist[dst] == INT_MAX ? -1 : dist[dst]
}.
```

Time Complexity

$O(k \times E)$

max no. of stops allowed.

Space Complexity

$O(V+E)$